Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL602**	**Course Name:**	**Cryptographic and system security Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**07**
Title of the Experiment:	**Study of packet sniffer tools wireshark, :- 1. Observer performance in promiscuous as well as non-promiscuous mode. 2. Show the packets can be traced based on different filters.**
Date of Performance:	**13/03/2026**
Date of Submission:	**13/03/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Ms Kshitija Gharat

Signature :

Date:

Aim: To student wireshark packet sniffer tool

Objectives:

- Identify security threats and malicious activity on a network
- Observe network traffic for debugging complex networks
- Filter traffic based on protocols, ports, and other parameters
- Capture packets and save them to a Pcap file for offline analysis
- Apply coloring rules to the packet list for better analysis

Theory:

Wireshark is a network packet analyzer. A network packet analyzer presents captured packet data in as much detail as possible.

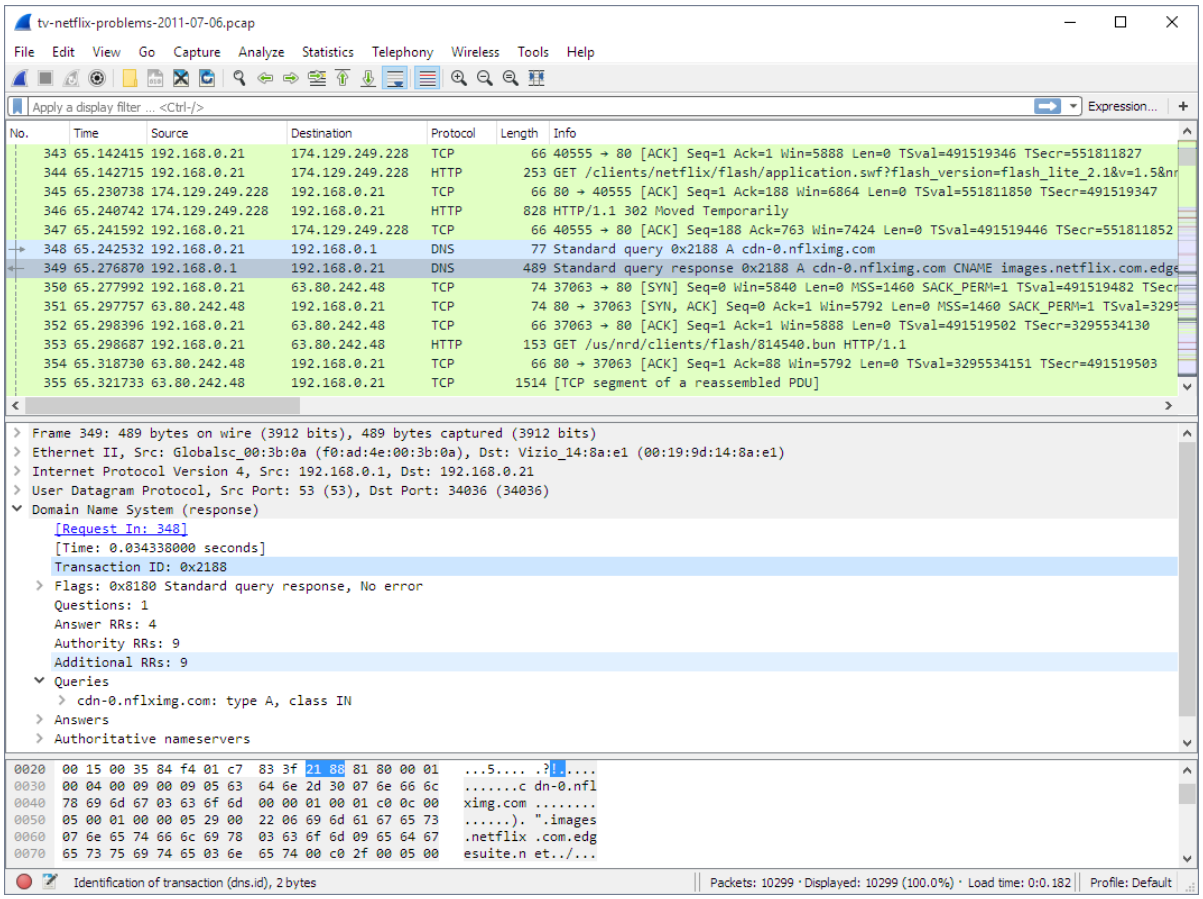
You could think of a network packet analyzer as a measuring device for examining what's happening inside a network cable, just like an electrician uses a voltmeter for examining what's happening inside an electric cable (but at a higher level, of course).

In the past, such tools were either very expensive, proprietary, or both. However, with the advent of Wireshark, that has changed. Wireshark is available for free, is open source, and is one of the best packet analyzers available today.

The following are some of the many features Wireshark provides:

- Available for *UNIX* and *Windows*.
- *Capture* live packet data from a network interface.
- *Open* files containing packet data captured with tcpdump/WinDump, Wireshark, and many other packet capture programs.
- *Import* packets from text files containing hex dumps of packet data.
- Display packets with *very detailed protocol information*.
- *Save* packet data captured.
- *Export* some or all packets in a number of capture file formats.
- *Filter packets* on many criteria.
- *Search* for packets on many criteria.
- *Colorize* packet display based on filters.

- Create various *statistics*.



1.1.3. Live capture from many different network media

Wireshark can capture traffic from many different network media types, including Ethernet, Wireless LAN, Bluetooth, USB, and more. The specific media types supported may be limited by several factors, including your hardware and operating system. An overview of the supported media types can be found at https://gitlab.com/wireshark/wireshark/-/wikis/CaptureSetup/NetworkMedia.

1.1.4. Import files from many other capture programs

Wireshark can open packet captures from a large number of capture programs. For a list of input formats see Section 5.2.2, "Input File Formats".

1.1.5. Export files for many other capture programs

Wireshark can save captured packets in many formats, including those used by other capture programs. For a list of output formats see Section 5.3.2, "Output File Formats".

1.1.6. Many protocol dissectors

There are protocol dissectors (or decoders, as they are known in other products) for a great many protocols: see Appendix C, Protocols and Protocol Fields.

1.1.7. Open Source Software

Wireshark is an open source software project, and is released under the GNU General Public License (GPL). You can freely use Wireshark on any number of computers you like, without worrying about license keys or fees or such. In addition, all source code is freely available under the GPL. Because of that, it is very easy for people to add new protocols to Wireshark, either as plugins, or built into the source, and they often do!

Wireshark should support any version of Windows that is still within its extended support lifetime. At the time of writing this includes Windows 11, 10, Server 2022, Server 2019, and Server 2016. It also requires the following:

The Universal C Runtime. This is included with Windows 10 and Windows Server 2019 and is installed automatically on earlier versions if Microsoft Windows Update is enabled. Otherwise you must install KB2999226 or KB3118401.

Any modern 64-bit Intel or Arm processor.

500 MB available RAM. Larger capture files require more RAM.

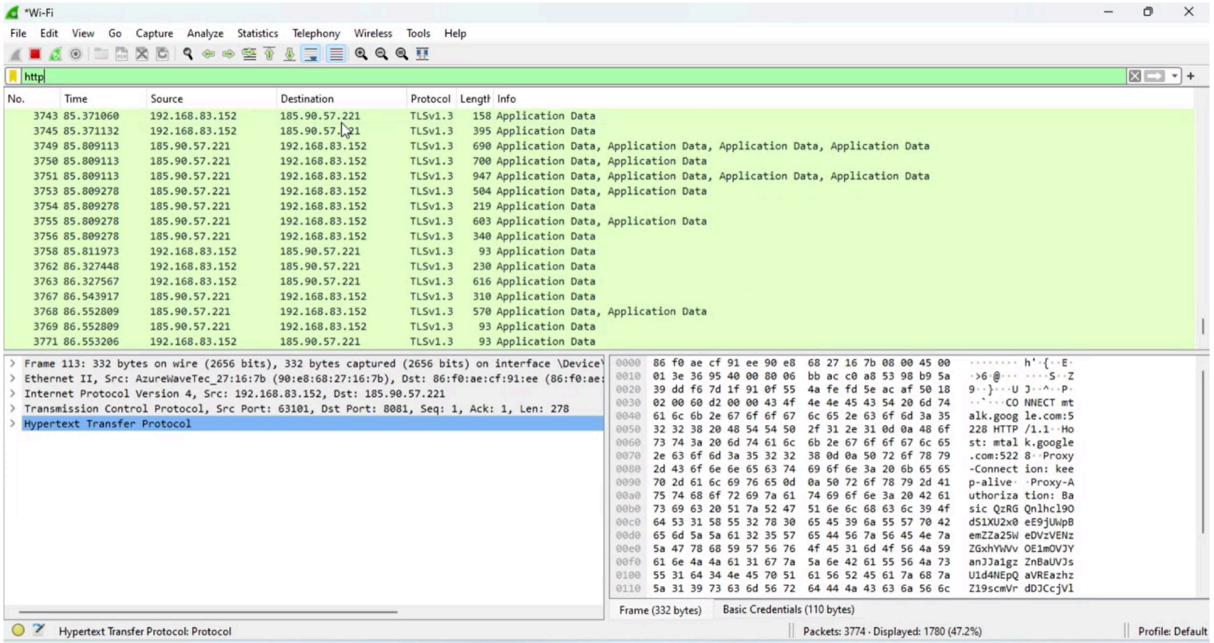
500 MB available disk space. Capture files require additional disk space.

Any modern display. 1280 × 1024 or higher resolution is recommended. Wireshark will make use of HiDPI or Retina resolutions if available. Power users will find multiple monitors useful.

A supported network card for capturing

- Ethernet. Any card supported by Windows should work. See the wiki pages on Ethernet capture and offloading for issues that may affect your environment.
- 802.11. See the Wireshark wiki page. Capturing raw 802.11 information may be difficult without special equipment.
- Other media. See https://gitlab.com/wireshark/wireshark/-/wikis/CaptureSetup/NetworkMedia.

Implementation:



The implementation of the Wireshark packet sniffer tool involves capturing, filtering, and analyzing network traffic to understand communication between devices. Initially, Wireshark is installed along with the required packet capture library (Npcap), which enables access to network interfaces. After launching the tool, the active network interface such as Wi-Fi or Ethernet is selected to begin capturing live packet data.

Once the capture process starts, Wireshark continuously records packets transmitted and received over the network. To generate meaningful traffic, network activities such as browsing websites or sending ping requests are performed. The captured packets are displayed in real-time, showing essential details like source and destination IP addresses, protocols used, and packet length.

To simplify analysis, filters are applied to isolate specific types of traffic such as HTTP, DNS, or TCP packets. These filters help in focusing on relevant data and identifying patterns or anomalies in the network. Each packet can be examined in detail, including its headers and payload, providing deep insights into how data is structured and transmitted.

Additionally, coloring rules are applied to visually distinguish between different types of packets, making the analysis more intuitive. Wireshark also provides statistical tools such as protocol hierarchy and traffic summaries to better understand network behavior.

Conclusion:

The study and implementation of Wireshark provided practical exposure to real-time network monitoring and packet analysis. Through this tool, we were able to capture live network traffic, filter packets based on protocols and IP addresses, and analyze detailed packet-level information.